Module 5 — MCP Servers

Lesson 5.1

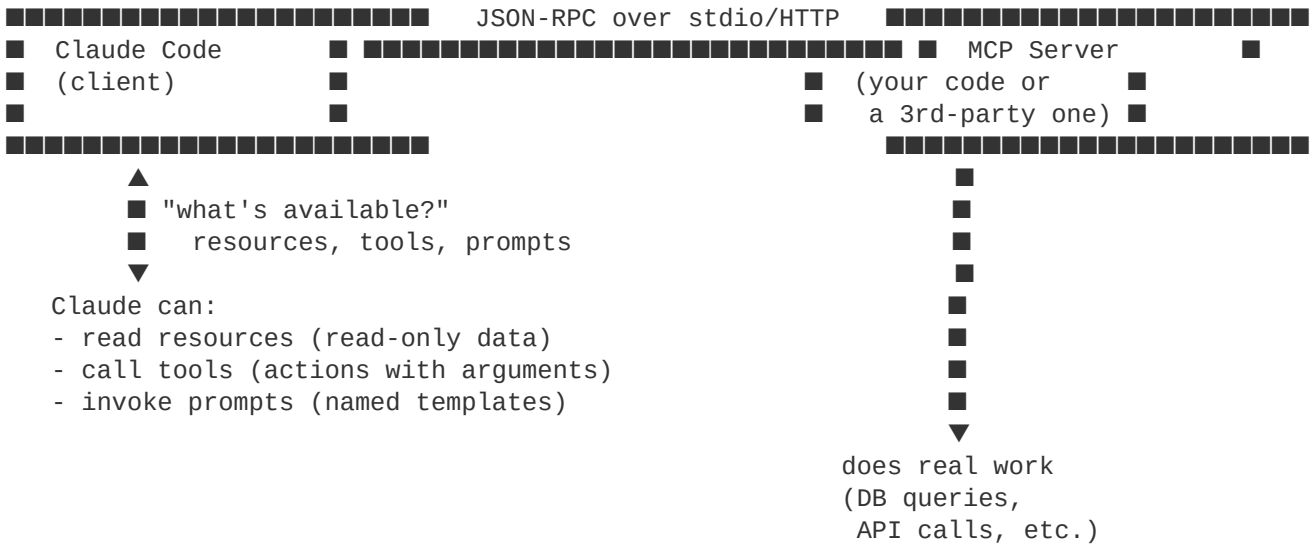
MCP overview

Model Context Protocol is a standard way for AI clients to talk to external tools and data. Claude Code is an MCP client; MCP servers expose capabilities Claude can call.

Why it matters

Slash commands and hooks customize Claude inside the project. MCP extends Claude *outside* the project — into your databases, your APIs, your design tools, your ticket tracker. If a system has structured data or actions, an MCP server can make them callable from Claude.

The mental model



The protocol is JSON-RPC. Transports are typically:

- **stdio** — the server is a subprocess Claude spawns. Most common for local tools.
- **HTTP / SSE** — the server runs over the network. Used for shared or remote servers.

The three MCP primitives

An MCP server exposes some combination of three things:

Primitive	Read or write?	Example
Resources	Read	A document, a row from a DB, a file in S3
Tools	Write/action	"Create a Linear ticket", "run a SQL query", "post to Slack"
Prompts	Templates	"Generate a release note", "draft a PR description"

Most servers offer tools; resources and prompts are less common but powerful when used right. We'll dig into the distinction in 5.4.

What you can do with MCP

- Talk to your bug tracker (Linear, Jira, GitHub Issues) from Claude
- Query your databases directly

- Fetch design files from Figma / Canva
- Hit internal APIs without writing a one-off curl every time
- Expose company-specific data (employee directory, on-call schedule) as resources
- Wrap any CLI tool as a tool

The MCP server ecosystem is growing fast — many official and community servers exist for common services.

When to use MCP vs. other options

Goal	Right tool
Add a one-off shell command Claude can run	Allow it in `permissions`
Add a reusable team workflow	Custom slash command
Enforce always-runs-on-X behavior	Hook
Connect Claude to a system with structured data/actions	MCP server
Build a programmatic agent (no Claude Code UI)	Agent SDK

If you find yourself writing more than 100 lines of shell glue to expose something to Claude, write an MCP server instead.

MCP and Claude Code

Claude Code is one of many MCP clients. Others include the Claude desktop apps, the Claude Agent SDK, and third-party tools. A server you write works with all of them — that's the point of a standard.

In Claude Code, MCP servers are configured per-project (in `settings.json`) or per-user (in `~/.claude/settings.json`). They load at session start.

What we'll build in this module

- 5.2: hook up existing MCP servers (the easy path)
- 5.3: build a minimal Python MCP server from scratch
- 5.4: the three primitives in depth
- 5.5: keep secrets out of trouble

Runnable example: examples/04-mcp-server-python.

Try it

1. Skim the official MCP servers list for one that fits your stack.
2. Note one workflow at work that's currently "open another tab, copy/paste into Claude." That's an MCP candidate.

Gotchas

- **MCP servers load at session start.** Changes require restarting `claude`.
- **stdio servers are subprocesses of `claude`.** They die when the session ends. State is in-memory unless you persist.
- **HTTP MCP servers need auth.** We cover this in 5.5. Don't expose tools without it.
- **MCP tools count as "tools" for permissions.** They show up in `/permissions` and respect allow/ask/deny rules.

Further reading

- MCP spec: https://modelcontextprotocol.io
- Community servers: https://github.com/modelcontextprotocol/servers
- Official Claude Code MCP docs: https://docs.claude.com/en/docs/claude-code/mcp

Next

→ 5.2 Connecting existing MCP servers